

IEEE x MercerX IC Design Workshop

Submit your design to get it fabricated for free!

3/29/26

RPI ECSE – Theresa O'Connor, Anvita Nagarkar



Rensselaer

why not change the world?®

Important Links

Attendance Form



SCAN ME

Discord Link

SCAN ME



Agenda

- Team Introductions
- Verilog to Silicon Workflow
- Intro to Verilog
- Verilog Statements and Blocks
- Coding Guidelines
- Walkthrough
- Github Repo

Theresa O'Connor, **Workshop Lead** CSE '27

Concentration:
Micro/Nanoelectronics

Email: oonnt8@rpi.edu

Discord: reesespiecezz



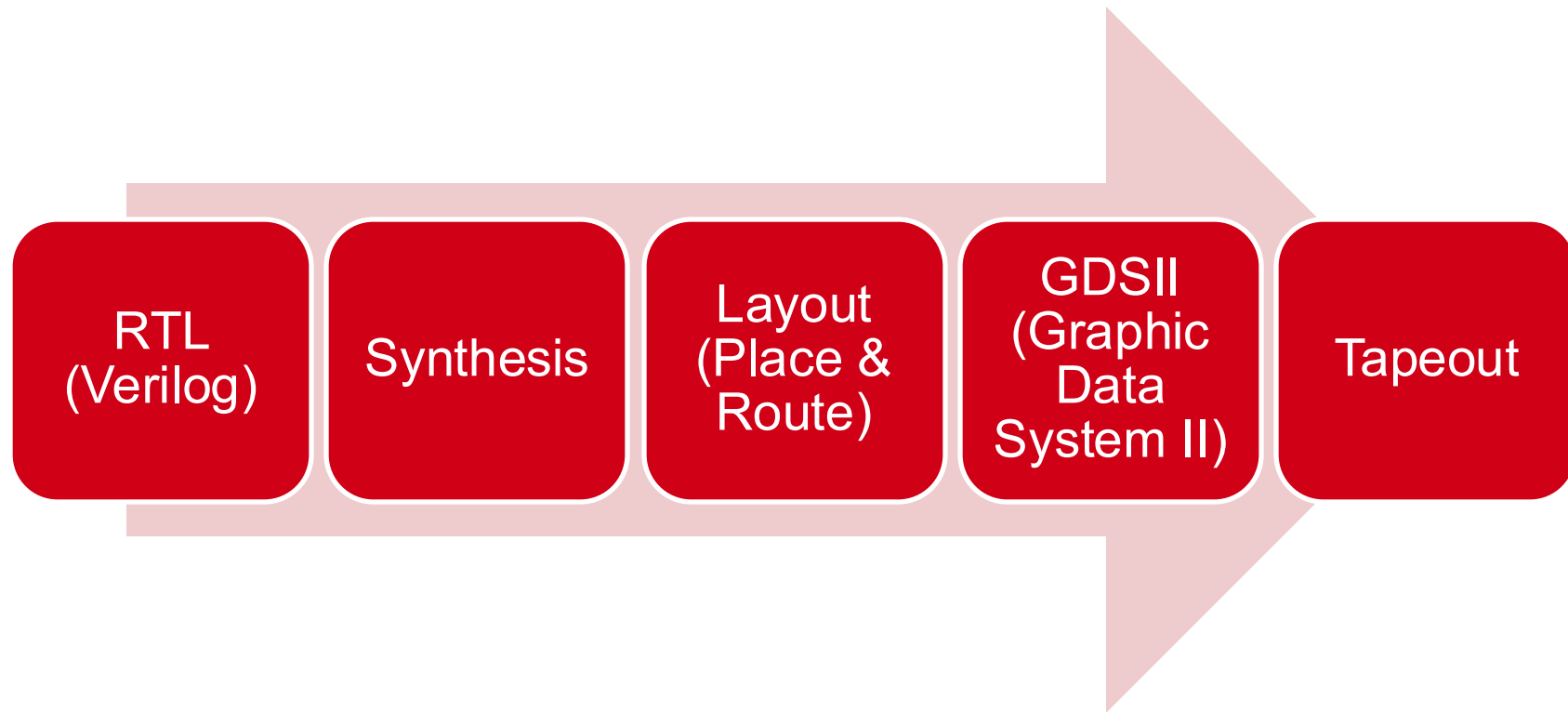
Anvita Nagarkar, **Workshop Assistant** EE '29

Email: nagara2rpi.edu

Discord: anvita.com



From Verilog to Silicon



Intro to Verilog

What is Verilog?

- Hardware Description Language (HDL)
- Describes behavior and structure of hardware
- Used for simulation, synthesis, and verification
- Tutorial:
<https://www.chipverify.com/tutorials/verilog> OR
<https://mercexlab.notion.site/verilog-crossy>

Verilog Modules

- Module = building block (like a function in software, but maps to hardware)
- Defines inputs, outputs, and internal logic
- Can be instantiated inside other modules

```
1 module <name> ([port_list]);  
2     // Contents of the module  
3 endmodule  
  
4  
5 // A module can have an empty portlist  
6 module name;  
7     // Contents of the module  
8 endmodule
```

Declaring Module I/O

- Inputs: signals coming into the module
 - input a, clk, rst;
- Outputs: signals leaving the module
 - output reg led;

```
1 // "dff" is the name of this module
2
3 module dff ( input  d,          // Inputs to the design should start with "input"
4             rstn,
5             clk,
6             output q);        // Outputs of the design should start with "output"
```


Verilog Coding Blocks

Data Types

- Nets (wire): represent physical connections
- Registers (reg): hold state across clock cycles
- Vectors: [N:0] for multi-bit buses
- Operators: +, -, &, |, ^, ~, <<, >>

Combinational Logic

- Continuous assignment: assign y = a & b;
- Procedural block: always @(*) begin ... end
- Must fully specify outputs in all conditions

Sequential Logic

- Flip-flops: always @(posedge clk)
- Registers updated on clock edge
- Reset handling (synchronous or asynchronous)

Declaring Internal Signals

- Declared after the IO declaration and before the endmodule statement
- wire → for combinational connections
- reg → for values stored in flip-flops (sequential)
- vectors → [N:0] defines multi-bit signals

```
1  module design;
2      wire    abc;
3      wire    a;
4      wire    b;
5      wire    c;
6
7      wire    abc;    // Error: Identifier "abc" previously declared
8
9      assign abc = a & b | c;
10 endmodule
```

Combinational Logic

- Logical operations have different symbols than gates
- Continuous assignment LHS is updated whenever any RHS variable is updated (logic gates)

Operator	Description	Operator	Description
		$a + b$	a plus b
$a < b$	a less than b	$a - b$	a minus b
$a > b$	a greater than b	$a * b$	a multiplied by b
$a \leq b$	a less than or equal to b	a / b	a divided by b
$a \geq b$	a greater than or equal to b	$a \% b$	a modulo b
		$a ** b$	a to the power of b

Operator	Type
$\&$	And
$\sim\&$	Nand
$ $	Or
$\sim $	Nor
$\wedge$	Xor
$\sim\wedge$	Xnor

Always Block

- Statements inside are executed sequentially
- Procedural always @(*) block - *Combinational*
 - * means that at any variable change, statements inside are executed
- always @(posedge clk) – *Sequential*
- Registers updated at each rising edge
- Can be used to implement flip-flops
- Use "<=" non-blocking assignment if multiple statements should occur simultaneously at the clock edge

```
1  always @ (event)
2      [statement]
3
4  always @ (event) begin
5      [multiple statements]
6  end
```

```
1  module jk_ff ( input      j,           // Input J
2                  input      k,           // Input K
3                  input      rstn,        // Active-low async reset
4                  input      clk,         // Input clk
5                  output reg q);          // Output Q
6
7      always @ (posedge clk or negedge rstn) begin
8          if (!rstn) begin
9              q <= 0;
10         end else begin
11             q <= (j & ~q) | (~k & q);
12         end
13     end
14 endmodule
```

If Statements

- Used within always blocks for conditional logic

```
1  if ([expression])
2      Single statement
3
4  // Use "begin" and "end" blocks for more than 1 statements
5  if ([expression]) begin
6      Multiple statements
7  end
```

```
17 if ([expression 1])
18     Single statement
19 else if ([expression 2]) begin
20     Multiple Statements
21 end else
22     Single statement
```

```
9  // Use else to execute statements for which expression is false
10 if ([expression]) begin
11     Multiple statements
12 end else begin
13     Multiple statements
14 end
```

Case Statements

- Useful for multiplexers, FSMs, and decoders
- Always include default for synthesis safety
- Expression will be evaluated once, options will be evaluated in order
 - Can use inside always blocks to evaluate each time the block runs

```
2  case (<expression>)
3      case_item1 :    <single statement>
4      case_item2,
5      case_item3 :    <single statement>
6      case_item4 :    begin
7                          <multiple statements>
8                          end
9      default      : <statement>
10 endcase
```

```
7      always @ (a, b, c, sel) begin
8          case(sel)
9              2'bxz      :    out = a;
10             2'bzx      :    out = b;
11             2'bxx      :    out = c;
12             default    :    out = 0;
13         endcase
14     end
```

Instantiation

- Instantiation: using one module inside another
- Used to achieve well defined hierarchical design
- Connections to and from the instantiated module must be defined on the same level as the module

```
1 module mydesign ( input x, y, z,  
2                   output o);  
3  
4 endmodule
```

```
1 module design_top;  
2     wire [1:0] a;  
3     wire      b, c;  
4  
5     mydesign d0 ( .x (a[0]),    // signal "x" in mydesign should be connected to "a[0]" in this module  
6                  .y (b),       // signal "y" in mydesign should be connected to "b" in this module (d  
7                  .z (a[1]),  
8                  .o (c));  
9 endmodule
```

Top-Level Module

- Once we have designed all the individual subcircuits within our system, we can combine them together via a "top" module
- Entry point of your design
- Defines all I/O pins that connect to outside world (TinyTapeout harness, FPGA, or testbench)
- Only one top-level module per project

```
1  module design_top;
2      mydesign d0 ( .x (a[0]),
3                  .z (a[1]),      // z at second position is okay because of explicit connection
4                  .y (a[1]),
5                  .x (b),         // illegal - x is already connected to a[0]
6                  .o (c));
7  endmodule
```


Verification

- To verify, write a testbench (tb)
- This provides a way to simulate the behavior of the design under various conditions and inputs
- Declare top-level testbench module
- Declare signals for DUT (Design Under Test) connection
- Create a module instantiation of the DUT module
- Connect testbench signals to the DUT ports (drive inputs with stimulus)
- Initialize test (initial block)
- Generate clk to sequence tests
- Check results

Example Testbench

```
80 module testbench();
1
2   logic      clk;
3   logic      reset;
4
5   logic [31:0] WriteData, DataAdr;
6   logic      MemWrite;
7
8   // instantiate device to be tested
9   top dut(clk, reset, WriteData, DataAdr, MemWrite);
10
11  // initialize test
12  initial
13  begin
14    reset <= 1; # 22; reset <= 0;
15  end
16
17  // generate clock to sequence tests
18  always
19  begin
20    clk <= 1; # 5; clk <= 0; # 5;
21  end
22
23  // check results
24  always @(negedge clk)
25  begin
26    if(MemWrite) begin
27      if(DataAdr === 100 & WriteData === 12) begin
28        $display("Simulation succeeded");
29        $stop;
30      end else if (DataAdr !== 96) begin
31        $display("Simulation failed");
32        $stop;
33      end
34    end
35  end
36 endmodule
37
38 ///////////////////////////////////////////////////
39 // End Testbench
40 ///////////////////////////////////////////////////
```



Verilog Coding Guidelines

- ✓ Use non-blocking ($\leq$) for sequential logic
- ✓ Use blocking ($=$) for combinational logic
- ✓ Always include a reset for registers/sequential logic
- ✓ Always define default values if using case or if/else
- ✓ Cover all branches in if/case
- ✓ Keep a single clock domain (for TinyTapeout)
- ✓ Explicitly declare reg and wire types
- ✓ Use parameters for constants (not define)→ makes code reusable
- ✓ Keep designs simple and modular

Verilog Coding Guidelines

- ✗ Don't use `#delay`, `initial`, or `$display` in synthesizable code (simulation only)
- ✗ Don't forget a default in case or if/else statements
- ✗ Don't mix blocking and non-blocking in the same always block
- ✗ Don't rely on implicit signal declarations (wire by default)
- ✗ Don't create feedback loops (assign `a = a & b;`)
- ✗ Don't ignore width mismatches → truncation/extension can cause bugs
- ✗ Don't leave outputs unassigned in all paths → latch inference

Starting a New Project

- Go to tinytapeout.com
- Open the advanced HDL link under Get Started
- Click HDL Templates, use the SKY shuttle template
- <https://github.com/TinyTapeout/ttsky-verilog-template>
- Make/Login to a GitHub account
- Click "Use this template" and "Create a new repository"
- Give your new repo a name and keep public visibility

Src → Project.v

- Top module
tt_um_example
- Input/Output pin vectors
- Bi-directional pin vectors
- Enable wire
- Clock wire
- Reset wire

Tool Installation

- Download WSL (Ubuntu recommended): <https://learn.microsoft.com/en-us/windows/wsl/install>
- In WSL bash: run "sudo apt update" and "sudo apt install iverilog gtkwave"
 - iverilog → simulator/compiler
 - vvp → simulation runner
 - gtkwave → waveform viewer
- Use a text editor to write the module code (WSL, VSCode, etc.)

```
oconnt8@DESKTOP-IQF4QDN:~$ sudo apt update  
[sudo] password for oconnt8:
```

```
oconnt8@DESKTOP-IQF4QDN:~$ sudo apt install iverilog gtkwave
```

Text Editor Options

■ WSL Text Editor

- Command: `mkdir verilog_demo`
- Command: `cd verilog_demo`
- Command: `nano counter.v`
- Save with CTRL+O, exit with CTRL+X
- Verify the file is there using command: `ls`
- Command: `nano tb_counter.v`

```
oconnt8@DESKTOP-IQF4QDN:~$ cd verilog_demo
oconnt8@DESKTOP-IQF4QDN:~/verilog_demo$ |
```

■ Windows Editor

- VS Code, Notepad++, or Visual Studio Code Remote – WSL extension
- Open WSL home directory in Windows Explorer (e.g. \\wsl.localhost\Ubuntu\home\username) using command: `"explorer.exe ."`
- Save counter.v and tb_counter.v files directly into your WSL project folder



Network



wsl.localhost



Ubuntu



home



oconnt8



Writing the Modules

- Copy + paste in the GitHub template
- We will do a walkthrough of a simple counter design
- Counter increments on each clock edge, or resets to 0
 - Internal register
 - Sequential logic (always block)
 - Assign output
 - Tie off unused I/O pins
- Give your top module a unique name such as `tt_um_<yourname>_counter` for this example
- Remember that all outputs must be assigned (if unused, assign 0)

Side by Side Comparison

```
`default_nettype none

module tt_um_example (
    input wire [7:0] ui_in,    // Dedicated inputs
    output wire [7:0] uo_out,  // Dedicated outputs
    input wire [7:0] uio_in,   // IOs: Input path
    output wire [7:0] uio_out, // IOs: Output path
    output wire [7:0] uio_oe,  // IOs: Enable path (active high: 0=input, 1=output)
    input wire      ena,       // always 1 when the design is powered, so you can ignore it
    input wire      clk,       // clock
    input wire      rst_n      // reset_n - low to reset
);

// All output pins must be assigned. If not used, assign to 0.
assign uo_out  = ui_in + uio_in; // Example: ou_out is the sum of ui_in and uio_in
assign uio_out = 0;
assign uio_oe  = 0;

// List all unused inputs to prevent warnings
wire _unused = &{ena, clk, rst_n, 1'b0};

endmodule
```

```
6 `default_nettype none
7
8 module tt_um_oconnt_counter (
9     input wire [7:0] ui_in,    // Dedicated inputs
10    output wire [7:0] uo_out,   // Dedicated outputs
11    input wire [7:0] uio_in,    // IOs: Input path
12    output wire [7:0] uio_out,  // IOs: Output path
13    output wire [7:0] uio_oe,   // IOs: Enable path (active high: 0=input, 1=output)
14    input wire      ena,        // always 1 when the design is powered, so you can ignore it
15    input wire      clk,        // clock
16    input wire      rst_n       // reset_n - low to reset
17 );
18
19 // Internal register: 8-bit counter
20 reg [7:0] count;
21
22 // Sequential logic: counter increments on every clock
23 always @(posedge clk) begin
24     if (!rst_n) // Active-low reset
25         count <= 8'b0;
26     else
27         count <= count + 1;
28 end
29
30 // Assign counter to output
31 assign uo_out = count;
32
33 // Tie off unused IOs
34 assign uio_out = 0;
35 assign uio_oe  = 0;
36
37 // Prevent warnings for unused signals
38 wire _unused = &{ena, ui_in, uio_in, 1'b0};
39
40 endmodule
```

Above: Original.

Right: New

Writing the Testbench

- Clock: always #5 clk = ~clk; → produces 10ns period (100 MHz in simulation units)
- Reset: Active-low (rst_n) → start low to reset, then release
- Inputs: ui_in and uio_in are optional for a simple counter; here we drive them to 0
- Outputs: uo_out is the counter value; uio_out and uio_oe are tied in the wrapper, no need to drive
- VCD dump: creates tt_um_counter.vcd for waveform viewing in GTKWave
- Run using 3 WSL commands: iverilog -o sim counter.v tb_counter.v
- Command 2: vvp sim
- Command 3: gtkwave tt_um_counter.vcd

Editing the Repo

info.yaml

- Update project section accordingly
- Make sure to edit top_module: "tt_um_counter"
- If you have multiple source files, write their names under source_files:
- Add descriptive names for used I/O

Docs → info.md

- Update How it works
- Update How to test
- Updated External hardware
- Every section must be changed from the default

Src .v Files

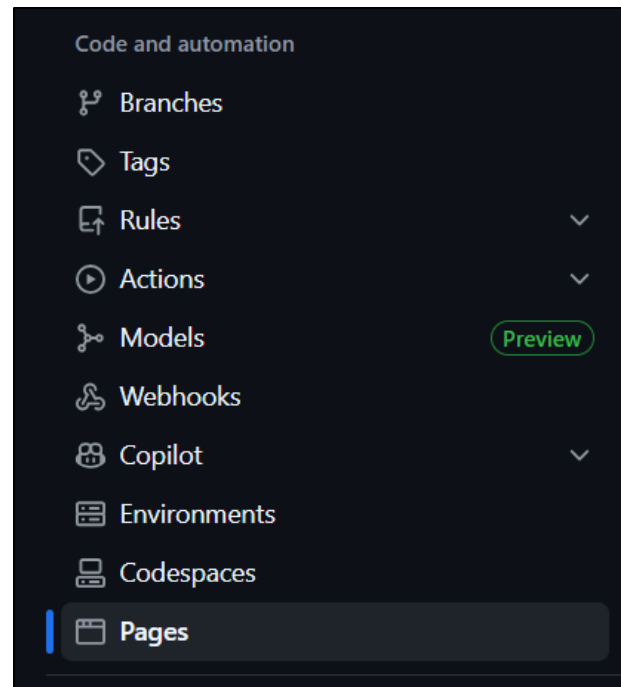
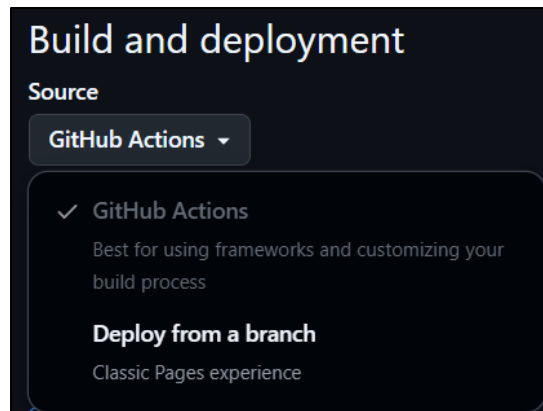
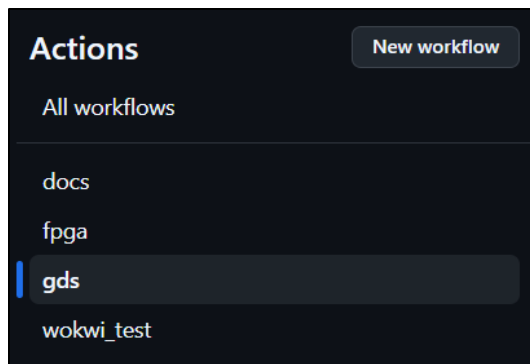
- Replace Project.v
- Upload any submodule files you created
- Test → Makefile: Update PROJECT_SOURCES if needed
- Test → tb.v: replace tt_um_example with tt_um_counter dut and write in the testbench
- Test → test.py: Change the assert line or comment out

Counter test.py

```
4 import cocotb
5 from cocotb.clock import Clock
6 from cocotb.triggers import ClockCycles, RisingEdge
7
8 @cocotb.test()
9 ✓ async def test_counter(dut):
10     dut._log.info("Start counter test")
11
12     # Set the clock period to 10 ns (100 MHz)
13     clock = Clock(dut.clk, 10, units="ns")
14     cocotb.start_soon(clock.start())
15
16     # Apply reset
17     dut.ena.value = 1
18     dut.ui_in.value = 0
19     dut.uio_in.value = 0
20     dut.rst_n.value = 0
21     await ClockCycles(dut.clk, 2)
22     dut.rst_n.value = 1
23
24     # Check that counter starts from 0
25     await RisingEdge(dut.clk)
26     assert int(dut.uo_out.value) == 0, f"Expected 0, got {int(dut.uo_out.value)}"
27
28     # Check a few increments
29     for expected in range(1, 6):
30         await RisingEdge(dut.clk)
31         got = int(dut.uo_out.value)
32         assert got == expected, f"Expected {expected}, got {got}"
33     dut._log.info(f"Counter value = {got}")
34
```

Github Repo Configuration

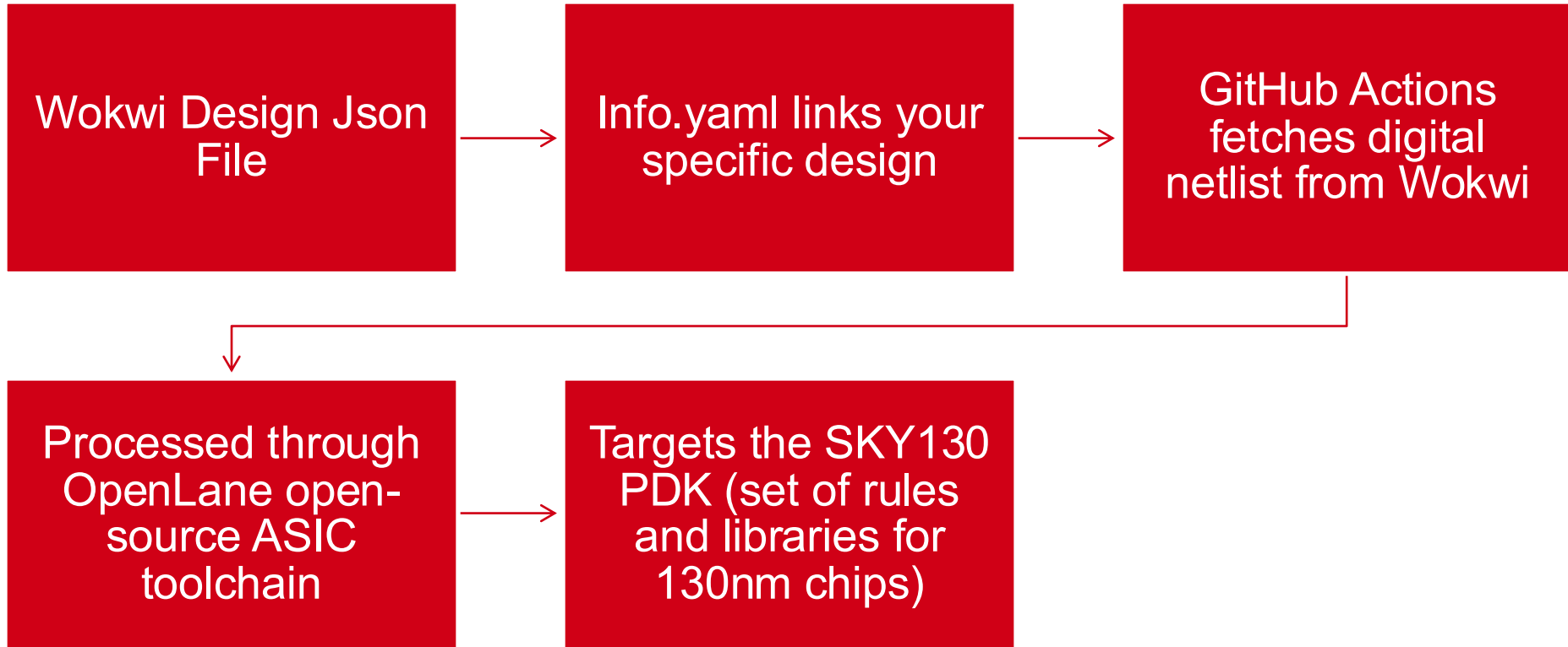
- Configure the repo to allow TinyTapeout's processes
- Go to Settings → Pages
- Under Build and Deployment Source, select Github Actions
- Go to Actions → gds and run workflow



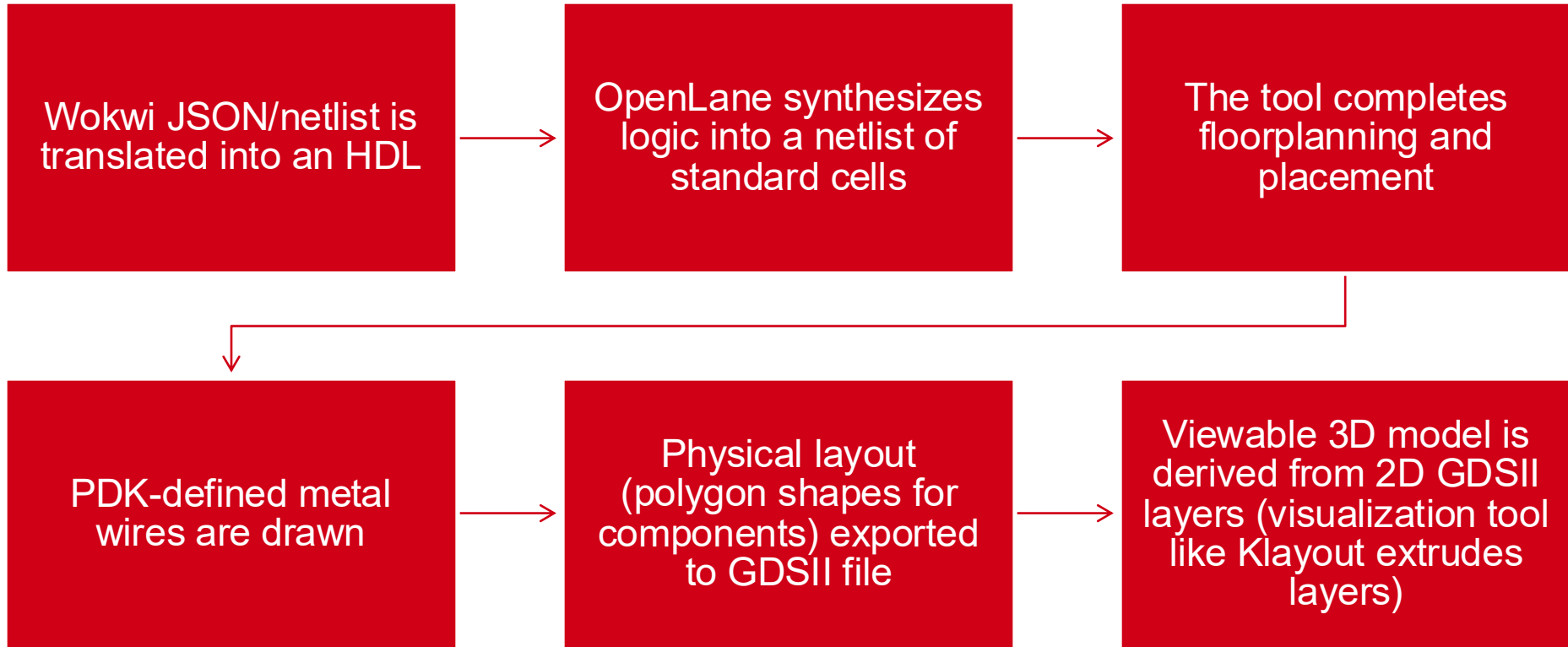
Github Repo 3D Viewer

- Click the most recent, successful gds workflow
- Scroll down the gds summary to the viewer summary
- Click "open 3D viewer" to see the layout of your design
- Key Layers:
 - Substrate – the base silicon “wafer”
 - Nwell – regions doped to host PMOS transistors (the “wells” where devices sit)
 - Diffusion (diff) – areas where source and drain terminals of transistors are formed
 - Polysilicon (poly) – narrow lines that cross diffusion to form transistor gates
 - Contacts (lcon, mcon, via) – tiny plugs that connect one layer to the next (e.g. diffusion → metal, metal1 → metal2, etc.)
 - Metal layers (met1, met2, ... met5) – wiring layers stacked above the devices; higher metals are thicker and used for longer connections or power

Github to Tapeout



Github to GDSII





Rensselaer

why not change the world?®